

Tencent Sparking Program 2026: Quantum Computing

Tencent Quantum Lab
(Confidential — Do NOT Distribute)

There will be two groups. Each group is required to complete Problem 1 and either Problem 2 or Problem 3. Each group should write its answers in a well-formatted document (e.g., Overleaf, Jupyter Notebook, or Tencent Docs). The final report, in the form of a slide deck, should be submitted jointly by the two groups after discussion. This report should clearly present your findings, methods, and conclusions. Crucially, it should consolidate the outputs from both groups, identify a common theme, and present a coherent discussion centered on the key discoveries related to that theme. Ensure that all members contribute to the report and that it reflects a collaborative effort.

Problem 1: Preliminaries and Warm-up

In this problem, you will review several fundamental concepts in quantum computing, including quantum states, quantum gates, quantum measurement, and quantum circuits. These concepts form the basic language for describing and implementing quantum algorithms. Through a series of simple exercises, you will become familiar with the basic mathematical and circuit-based descriptions of quantum computation. This problem is designed as a warm-up for the later tasks on quantum algorithms.

We introduce two basic circuit-resource measures: circuit size and circuit depth. The circuit size is the total number of gates in a circuit, while the circuit depth is the minimum number of sequential gate layers after gates acting on disjoint qubits are parallelized. We will also introduce ancillary qubits, i.e., additional workspace qubits used to implement quantum computations. They are typically initialized in the state $|0\rangle$ and returned to $|0\rangle$ by the end of the computation.

1. Show that $R_n(\theta) = e^{-i\frac{\theta}{2}\mathbf{n}\cdot\boldsymbol{\sigma}}$ takes the form

$$\cos(\theta/2)I - i\sin(\theta/2)(n_X X + n_Y Y + n_Z Z), \quad (1)$$

where $\mathbf{n}$ is a unit 3D vector and $\boldsymbol{\sigma} = (X, Y, Z)$.

2. Show that any single-qubit unitary U_2 takes the form $e^{i\alpha}R_n(\theta)$ for some α , $\mathbf{n}$, and θ .
3. Implement $R_n(\theta)$ using $R_Y(\alpha)$ and $R_Z(\beta)$ for appropriate choices of α and β .

4. State Preparation

Starting with all qubits initialized in $|0\rangle$, prepare the following states using single-qubit gates, two-qubit gates, ancillary qubits, classical feedback, and measurements. The resource cost of the final circuit should be as small as possible. You may optimize for circuit size, circuit depth, or both simultaneously.

- (a) Prepare an n -qubit quantum state $\frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} |x\rangle$;
- (b) Prepare an n -qubit state $\alpha |000 \cdots 000\rangle + \sqrt{1 - \alpha^2} |111 \cdots 111\rangle$ for any $\alpha \in \mathbb{R}$;
- (c) Prepare the 4-qubit state $\frac{3}{10} |0001\rangle + \frac{2}{5} |0010\rangle + \frac{\sqrt{6}}{4} |0100\rangle + \frac{\sqrt{6}}{4} |1000\rangle$;
- (d) (Optional) Prepare the following state:

$$\begin{aligned} & \alpha(|1000000\rangle + |0010101\rangle + |1110011\rangle + |0100110\rangle \\ & \quad + |1001111\rangle + |0011010\rangle + |1111100\rangle + |0101001\rangle) \\ & + \beta(|0111111\rangle + |1101010\rangle + |0001100\rangle + |1011001\rangle \\ & \quad + |0110000\rangle + |1100101\rangle + |0000011\rangle + |1010110\rangle) \end{aligned}$$

- (e) (Optional) Use TensorCircuit to verify your answers.

5. Gate Construction

- (a) Use CNOT gates to implement the SWAP operation: $\text{SWAP} |\psi\rangle |\phi\rangle = |\phi\rangle |\psi\rangle$.
- (b) Consider four qubits q_0, q_1, q_2 and q_3 . Implement two CNOT gates between q_0 and q_2 , and q_1 and q_3 respectively, under the constraint that only CNOT gates between q_i and q_{i+1} are available. Try to use as few gates as possible.
- (c) Use CNOT and R_Z gates to implement the two-qubit gate R_{ZZ} , where

$$R_Z(\theta) = e^{-i\theta/2Z}, \quad R_{ZZ}(\theta) = e^{-i\theta/2Z \otimes Z}.$$

- (d) Construct a Controlled-Phase gate

$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & e^{i\theta} \end{pmatrix} \quad (5)$$

using R_{ZZ} and single-qubit gates.

- (e) For an arbitrary n -qubit Pauli operator P , construct a circuit to implement $e^{-i\theta P}$.
- (f) Implement the following specific $\text{SU}(4)$ gate:

$$\begin{bmatrix} \frac{1}{2} & \frac{1}{\sqrt{2}} & \frac{1}{2} & 0 \\ \frac{1}{2} & -\frac{1}{\sqrt{2}} & \frac{1}{2} & 0 \\ \frac{1}{2\sqrt{2}} & 0 & -\frac{1}{2\sqrt{2}} & \frac{\sqrt{3}}{2} \\ \frac{\sqrt{3}}{2\sqrt{2}} & 0 & -\frac{\sqrt{3}}{2\sqrt{2}} & -\frac{1}{2} \end{bmatrix}$$

using single-qubit pulses and CZ gates.

- (g) (Optional) Use TensorCircuit to verify your answer above.

6. Measurement

- (a) Construct a quantum circuit to measure the two-qubit operators $Z_1 Z_2$ and $X_1 X_2$ on $|\psi\rangle$ and project $|\psi\rangle$ onto the joint $+1$ eigenstate of $Z_1 Z_2$ and $X_1 X_2$.
- (b) Give a general method to construct a quantum circuit to perform projective measurement of any Pauli operator P .

- (c) Prepare the single-qubit quantum state $\alpha|0\rangle + \sqrt{1-\alpha^2}|1\rangle$ from an n -qubit quantum state $\alpha|000\cdots 000\rangle + \sqrt{1-\alpha^2}|111\cdots 111\rangle$ by measurement.
- (d) (Optional) A 5-qubit fanout gate F_4 acting on qubits q_0, q_1, q_2, q_3, q_4 is defined as follows

$$F_4|x_0\rangle|x_1, x_2, x_3, x_4\rangle = |x_0\rangle|x_1 \oplus x_0, x_2 \oplus x_0, x_3 \oplus x_0, x_4 \oplus x_0\rangle,$$

where $x_0, x_1, \dots, x_4 \in \{0, 1\}$. Two-qubit gates may act only on qubits (q_j, q_{j+1}) for $0 \leq j \leq 3$. Construct the quantum circuit of F_4 to minimize the circuit depth and size. (Hint: use ancillary qubits and measurements.)

- (e) (Optional) Use CNOTs and Hadamards to compute the inner product $|\langle\psi|\phi\rangle|$ for arbitrary states $|\psi\rangle$ and $|\phi\rangle$.

7. Reversible Circuit

(Optional) Construct a reversible classical circuit using Toffoli and NOT gates to evaluate the formula:

$$S_3(x) = (\neg x_1 \vee \neg x_2) \wedge (x_1 \vee \neg x_2) \wedge (\neg x_1 \vee x_2 \vee \neg x_3) \wedge (\neg x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee x_3),$$

where each x_i represents a Boolean value, and $\neg$, $\vee$, and $\wedge$ denote logical NOT, OR, and AND, respectively. Try to: (1) use as few ancillary qubits as possible, and (2) minimize the product of the number of gates and the total number of qubits.

8. Grover Search

(Optional) Using the results from the previous problem, find a configuration of x satisfying $S_3(x) = \text{True}$ with Grover's search algorithm. Implement your algorithm using TensorCircuit and verify the solution.

Problem 2: Breaking the Limits of SQD — From Quantum Circuits to EWF Embedded Wave Functions

Sample-based Quantum Diagonalization (SQD) is a quantum-classical hybrid algorithm that solves molecular ground-state energies by sampling bitstrings from a quantum state and exactly diagonalizing the Hamiltonian in the sampled subspace. However, SQD alone is limited by molecular size — the Full CI (FCI) space grows exponentially with orbital count. The Embedded Wave Function (EWF) method breaks this limit by splitting large molecules into small fragments, solving each with high precision, and recombining the results.

In this problem, you will start from SQD basics on H_2 , understand accuracy control and LUCJ sampling improvement, then transition to EWF embedding theory — Schmidt decomposition, bath construction, DMET vs EWF differences, fragment solver selection — and ultimately use the full EWF-SQD pipeline to solve the water molecule ground-state energy.

Notation: n = number of spatial orbitals, N_e = number of electrons, N_α , N_β = number of α , β spin electrons. h_{pq} = one-body integrals, $(pq|rs)$ = two-body integrals (chemist’s notation). $\hat{a}_p^\dagger$, $\hat{a}_p$ = fermionic creation/annihilation operators. $\hat{n}_p = \hat{a}_p^\dagger \hat{a}_p$ = number operator. S = number of samples, M = number of unique configurations after deduplication.

Consider five test molecules:

- (a) H_2 : STO-3G, 2 spatial orbitals, 2 electrons, bond length 0.74 Å (4 qubits);
- (b) LiH: STO-3G, 4 spatial orbitals, 4 electrons (2 frozen core), bond length 1.55 Å (8 qubits);
- (c) H_2O : STO-3G, 7 spatial orbitals, 10 electrons (2 frozen core), O-H 0.96 Å, H-O-H 104.5° (14 qubits);
- (d) N_2 : STO-3G, 10 spatial orbitals, 14 electrons (4 frozen core), bond length 1.1 Å (20 qubits, triple bond, strongly correlated);
- (e) C_2H_4 : STO-3G, 14 spatial orbitals, 16 electrons, C-C 1.33 Å, C-H 1.08 Å (28 qubits).

Show numerically and analytically that SQD and EWF-SQD can be used to solve the ground-state energy of these molecules. Please do the following using PySCF and TensorCircuit:

1. Build Hamiltonian and Mapping Comparison

To place a molecule on a quantum computer, one must first map the fermionic Hamiltonian to qubit Pauli operators. Jordan-Wigner (JW) and Parity are two common mappings — they are mutually exclusive alternatives. Closed-shell molecules possess N_α and N_β particle-number conservation symmetries ($\mathbb{Z}_2$ symmetries), each eliminating 1 qubit.

Given: Molecules (a) H_2 (STO-3G) and (d) N_2 (STO-3G).

Solve:

- (a) For H_2 , use PySCF to compute RHF and extract one-body integrals h_{pq} (p, q are spin-orbital indices) and two-body integrals $(pq|rs)$ (chemist’s notation, p, q, r, s are spin-orbital indices). Write the second-quantized Hamiltonian $\hat{H} = \sum_{pq} h_{pq} \hat{a}_p^\dagger \hat{a}_q + \frac{1}{2} \sum_{pqrs} (pq||rs) \hat{a}_p^\dagger \hat{a}_q^\dagger \hat{a}_s \hat{a}_r + E_{\text{core}}$.
- (b) Using $h_{01} \hat{a}_0^\dagger \hat{a}_1$ (subscripts 0,1 are spin-orbital indices) as an example, derive its Pauli string expansion under JW.

- (c) Count Pauli terms and maximum Pauli weight (non- I count) for H_2 under both mappings.
- (d) Use the two $\mathbb{Z}_2$ symmetries ($N_\alpha=1$, $N_\beta=1$) to reduce H_2 from 4 to 2 qubits. Write the reduced Hamiltonian explicitly; explain how JW and parity differ in achieving this.
- (e) Repeat (c) for N_2 (20 qubits).

Advanced Challenge: BK mapping has Z-string length $O(\log n)$, hence max weight far lower than JW. For N_2 ($n=10$), what is BK's max weight? How many times lower is the circuit depth?

2. HF State and SQD Subspace Diagonalization

SQD samples S bitstrings from a quantum state $|\Psi\rangle$, each corresponding to an electronic configuration (Slater determinant $|D_k\rangle$), and exactly diagonalizes the CI matrix $H_{kl} = \langle D_k | \hat{H} | D_l \rangle$ in the subspace $\mathcal{S} = \text{span}\{|D_k\rangle\}$. The simplest initial state is the HF state $|\text{HF}\rangle = \prod_{i \in \text{occ}} \hat{a}_i^\dagger |0\rangle$ — a single computational basis state.

Given: Molecule (a) H_2 , 4 spin-orbitals (indexed 0–3), 2 electrons.

Solve: Determine which computational basis state $|b_3 b_2 b_1 b_0\rangle$ ($b_i \in \{0, 1\}$) H_2 's HF state corresponds to under JW, and write the preparation circuit. Manually list all $\binom{4}{2}=6$ configurations, build the 6×6 CI matrix, exactly diagonalize, and compare with PySCF FCI energy. If the subspace contains only HF + all single excitations (no doubles), what does the SQD energy equal? Explain why using Slater-Condon rules and HF self-consistency.

Advanced Challenge: Stretch H_2 bond to 3.0 Å. Does the conclusion about HF + singles still hold? What happens to HF self-consistency in the strongly correlated regime?

3. Configuration Recovery: Principle and Implementation

Quantum hardware noise causes sampled bitstrings to violate particle-number conservation. Configuration recovery uses the average occupancy $\bar{n}_i = \frac{1}{S} \sum_{s=1}^S b_i^{(s)}$ ($b_i^{(s)}$ = value of bit i in the s -th sampled bitstring) to correct erroneous bitstrings.

Given: H_2 's HF bitstring is $|0011\rangle$ (see Problem 2), noise flip probability 30%, sample count $S=1000$.

Solve:

- (a) Generate $S=1000$ noisy bitstrings by independently flipping each bit with 30% probability. Implement configuration recovery: for each violating bitstring, flip the bit with largest $|b_i - \bar{n}_i|$ until constraints satisfied. Verify correctness.
- (b) Derive: why is “flip the largest $|b_i - \bar{n}_i|$ ” optimal in the maximum likelihood sense?
- (c) Increase noise rate from 10% to 50%. Plot recovery success rate vs. noise rate. Determine the failure threshold (50% success rate) and explain the physics.

Advanced Challenge: Formulate as Bayesian inference — $\bar{n}_i$ as prior, observed bitstring as likelihood, find MAP configuration. How does this relate to (b)?

4. LUCJ Quantum Circuit Gate Decomposition

Sampling from HF yields only 1 configuration. The LUCJ ansatz uses CCSD amplitudes to build a correlated state. Its UCJ operator is $\hat{U}_{\text{UCJ}} = \hat{U} e^{i\hat{J}} \hat{U}^\dagger$, where $\hat{U} = e^{\hat{\kappa}}$ is orbital rotation ($\hat{\kappa}$ anti-Hermitian), $\hat{J} = \sum_{i,j} J_{ij} \hat{n}_i \hat{n}_j$ is diagonal Coulomb. The “local” truncation

retains only adjacent-qubit R_{ZZ} . Givens rotation = 2 CNOT + single-qubit gates; $R_{ZZ} = 2$ CNOT + 1 R_Z .

Given: Molecule (a) H_2 (1 UCJ layer); $n=20$ qubits on a 1D chain.

Solve:

- Derive how $e^{iJ_{ij}\hat{n}_i\hat{n}_j}$ maps to R_{ZZ} . Hint: use $\hat{n}_i = \frac{I-Z_i}{2}$.
- Build the complete LUCJ circuit for H_2 , count CNOTs and depth.
- For $n=20$ on 1D chain, use edge coloring to argue: full UCJ R_{ZZ} depth $O(n^2)$, LUCJ $O(1)$. Give specific values.

Advanced Challenge: On 2D heavy-hex topology, what is LUCJ's R_{ZZ} layer depth? Advantages over 1D?

5. LUCJ-SQD vs HF-SQD: How Entanglement Improves Sampling

The HF state is a computational basis state — measurement is deterministic, yielding only 1 configuration. The LUCJ state $|\Psi_{\text{LUCJ}}\rangle = \hat{U}_{\text{UCJ}} |\text{HF}\rangle$ contains CNOT gates creating entanglement, making measurement random and covering more configurations. Parameter `ccsd_scale` $\lambda \in [0, 1]$: $\lambda=0$ gives HF state.

Given: Molecule (b) LiH (STO-3G), $\lambda=0.05$, $S=1000$.

Solve:

- Sample $S=1000$ bitstrings from HF and LUCJ ($\lambda=0.05$) for LiH, feed into SQD, compare energy accuracy vs. FCI.
- Explain from quantum information perspective why LUCJ can cover more FCI configurations (key mechanism?), and discuss what $\lambda=0$ and $\lambda=1$ correspond to. Why use 0.05 instead of 1?

Advanced Challenge: Is there a quantitative relationship between LUCJ state's entanglement entropy and required samples S ?

6. SQD vs VQE: Quantum Resources and Energy Accuracy (Open)

VQE prepares $|\Psi(\theta)\rangle$ (p parameters), computes gradients via parameter-shift ($2p$ evaluations per step), each energy evaluation measures $O(N^4)$ Pauli terms with $O(1/\epsilon^2)$ shots per term. SQD prepares a fixed state, samples S bitstrings, iterates N_{iter} times. Current NISQ: $p_{2q} \sim 0.01$.

Given: Molecule (b) LiH ($N=8$). VQE: $p=4$, $N_{\text{iter}}=100$; SQD: $S=1000$, $N_{\text{iter}}=5$; $\epsilon=10^{-3}$ Ha.

Solve:

- Quantum resources:* Compute total circuit executions for both, compare orders of magnitude.
- Energy accuracy:* Given $|\Psi\rangle = \sum_x c_x |x\rangle$, define $\mathcal{S} = \text{span}\{|x\rangle : c_x \neq 0\}$. Prove $E_{\text{SQD}} = \min_{|\phi\rangle \in \mathcal{S}} \langle \phi | \hat{H} | \phi \rangle \leq \langle \Psi | \hat{H} | \Psi \rangle = E_{\text{VQE}}$. When does equality hold?
- Balanced comparison:* List advantages/disadvantages of SQD and VQE in: circuit depth, mapping flexibility, ansatz choice, noise robustness, measurement overhead, variational scope. Which is more practical on NISQ ($p_{2q} \sim 0.01$)? In the FTQC era ($p_{2q} < 10^{-4}$)?

Advanced Challenge: Design a hybrid — VQE’s shallow circuit for state prep, SQD’s diagonalization for post-processing. What is the energy lower bound?

7. FCI Space Explosion and EWF’s Scaling Advantage

FCI dimension $\binom{2n}{N_e}$ grows exponentially. EWF splits into F fragments, each with n_{frag} orbitals and N_{frag} electrons, solved independently.

Given: Test molecules (a)–(e). C_2H_4 ’s EWF max fragment: 26 qubits ($n_{\text{frag}}=13$).

Solve:

- Compute FCI dimensions for all 5 molecules, tabulate, observe trend.
- Prove: fixed n_{frag} , $F = N/n_{\text{frag}} \rightarrow$ EWF cost $O(N)$.
- Compute C_2H_4 max fragment FCI dimension, compare with full $\sim 3.1 \times 10^7$.
- Derive crossover: direct FCI $O(\binom{2N}{N_e}^3)$ vs. EWF $F \times O(\binom{2n_{\text{frag}}}{N_{\text{frag}}}^3)$. At what n_{frag} are they equal?

Advanced Challenge: Under 20-qubit NISQ constraint ($n_{\text{frag}} \leq 10$), optimal n_{frag} for N_2 and C_2H_4 ?

8. DMET Fragmentation: Schmidt Decomposition and Bath Exactness

EWF uses Schmidt decomposition: $|\Psi\rangle = \sum_{k=1}^r \lambda_k |u_k\rangle \otimes |v_k\rangle$ ($\lambda_k \geq 0$ Schmidt coefficients, r Schmidt rank), retaining n_{bath} largest- λ_k environment states as bath. DMET minimal bath ($n_{\text{bath}}=n_{\text{imp}}$) is exact at HF level (γ^{HF} is a projector: $\gamma^2=\gamma$), but becomes inexact when solver is upgraded.

Given: Molecule (c) H_2O , DMET atomic fragmentation.

Solve:

- Prove $r \leq \min(\dim \mathcal{H}_{\text{imp}}, \dim \mathcal{H}_{\text{env}})$, derive bath upper bound n_{imp} .
- Prove DMET minimal bath exactly reproduces HF density matrix. Key: projector property.
- When solver upgrades to CCSD/SQD, $\gamma^{\text{corr}} \neq (\gamma^{\text{corr}})^2$. Why is minimal bath no longer exact? What correlation does MP2 bath supplement?

Advanced Challenge: How is DMET’s self-consistent μ determined? Oneshot vs. self-consistent DMET trade-offs?

9. Energy Reconstruction: Algebraic Elimination of Double-Counting

Naive sum $\sum_f E_f$ is too large (overlapping bath HF contributions counted twice). Standard reconstruction: $E_{\text{total}} = E_{\text{MF}} + \sum_f (E_f - E_{\text{MF},f})$. Fragment energy decomposes: $E_f = E_{\text{core},f} + E_{\text{elec},f}$ (core = nuclear repulsion + frozen core, solver-independent; electronic = solver-dependent).

Given: Molecule (c) H_2O , fragments O and H_2 . Values (Ha): $E_{\text{MF}}=-74.963$, $E_{\text{O}}=-75.010$, $E_{\text{MF},\text{O}}=-74.980$, $E_{\text{H}_2}=-75.005$, $E_{\text{MF},\text{H}_2}=-74.983$.

Solve: Substitute $E_f = E_{\text{core},f} + E_{\text{elec},f}$ and $E_{\text{MF},f} = E_{\text{core},f} + E_{\text{HF-elec},f}$ into $(E_f - E_{\text{MF},f})$. Prove $E_{\text{core},f}$ cancels, leaving $\Delta E_{\text{corr},f} = E_{\text{elec},f} - E_{\text{HF-elec},f}$. Why is $\Delta E_{\text{corr},f}$ local and free from double-counting? Verify: HF fragments $\rightarrow E_{\text{total}} = E_{\text{MF}}$. Compute reconstructed total energy.

Advanced Challenge: Partitioned cumulant reconstruction — basic idea? When significantly more accurate?

10. Full Pipeline Integration and End-to-End Error Analysis (Open)

Chaining Problems 1–9 gives the complete EWF-SQD pipeline: RHF $\rightarrow$ DMET fragmentation $\rightarrow$ per-fragment LUCJ-SQD $\rightarrow$ energy reconstruction. Three error sources: ϵ_{frag} (bath truncation, decreases with larger fragments), ϵ_{SQD} (incomplete sampling, decreases with more samples S), ϵ_{noise} (quantum noise, related to circuit depth and p_{2q}).

Given: Molecules (a) H_2 , (c) H_2O , (d) N_2 , (e) C_2H_4 . Fixed budget B . Error model $\epsilon_{\text{total}} \approx \epsilon_{\text{frag}} + \epsilon_{\text{SQD}}$, cost model $B = c_1 N n_{\text{frag}}^3 + c_2 N S^3 / n_{\text{frag}}$.

Solve:

- (a) Run full pipeline on all 4 molecules, report E_{MF} , $E_{\text{EWF-SQD}}$, E_{FCI} (or CCSD).
- (b) Under fixed B , use Lagrange multipliers to derive optimal allocation $\epsilon_{\text{frag}} \approx \epsilon_{\text{SQD}}$ (“barrel effect”).
- (c) Discuss: H_2O (14q) $\rightarrow$ C_2H_4 (28q), which error grows fastest? N_2 bond stretched to 2.0 Å, which degrades fastest?

Advanced Challenge: In FTQC era, SQD $\rightarrow$ QPE. How does EWF reduce QPE resources from $O(N/\epsilon)$ to $O(n_{\text{frag}}/\epsilon)$? At what p_{2q} does QPE outperform SQD?

Optional problems.

1. Optimal ccscd_scale

Increasing `ccscd_scale` λ improves sampling coverage (LUCJ state deviates more from HF) but also increases UCJ diagonal Coulomb truncation error (non-adjacent R_{ZZ} dropped by “local” truncation $\propto \lambda^2$).

Given: Molecule (b) LiH , $\lambda \in [0, 0.5]$.

Solve: Vary λ from 0 to 0.5, plot LUCJ-SQD energy curve. Derive the existence of an optimal λ and explain the competition.

2. Quantum Noise and Configuration Recovery

Given: Molecule (b) LiH ’s LUCJ circuit has N_{2q} two-qubit gates and N_{1q} single-qubit gates, depolarizing noise $p_{2q} = 0.01$, $p_{1q} = 0.001$.

Solve: Derive $\epsilon_{\text{eff}} \approx (p_{2q} N_{2q} + p_{1q} N_{1q})/2$. Compare SQD energy with/without configuration recovery. Determine the noise threshold at which recovery fails.

3. EWF Bath Threshold

MP2 BNO occupation numbers decay roughly exponentially; truncation threshold η controls bath orbital count.

Given: Molecule (c) H_2O , $\eta \in [10^{-6}, 10^{-2}]$.

Solve: Derive asymptotic relationship n_{BNO} vs. η . Plot fragment size and energy error vs. η , find the inflection point, explain diminishing returns.

4. EWF on C_2H_4

Given: Molecule (e) C_2H_4 (STO-3G, 28 qubits). Two strategies: atomic (2 CH_2) and π/σ separation.

Solve: Derive which strategy has smaller inter-fragment entanglement. Run EWF-SQD to verify. Discuss: for C_6H_6 (72 qubits, max fragment 30) and CH_3COOH (88 qubits, max fragment 50), how does EWF transform “NISQ-infeasible” to “feasible”?

5. From SQD to QPE

SQD is a NISQ algorithm (depth $O(n)$), QPE is a fault-tolerant algorithm (depth $O(1/\epsilon)$), requires $\lceil \log_2(1/\epsilon) \rceil$ auxiliary qubits).

Solve: Derive differences in qubit count and depth. How does EWF reduce QPE resources? At what p_{2q} does QPE outperform SQD?

Note: Problems labeled (Open) are open-ended exploration problems. *Advanced Challenge* items are optional, encouraging deeper thinking.

Background: Key Concepts.

1. Molecular Electronic Hamiltonian and Second Quantization.

Under the Born-Oppenheimer approximation, the electronic Hamiltonian of a molecule with n spatial orbitals and N_e electrons is

$$\hat{H} = \sum_{pq} h_{pq} \hat{a}_p^\dagger \hat{a}_q + \frac{1}{2} \sum_{pqrs} (pq||rs) \hat{a}_p^\dagger \hat{a}_q^\dagger \hat{a}_s \hat{a}_r + E_{\text{core}}, \quad (1)$$

where $h_{pq} = \langle \phi_p | \hat{T} + \hat{V}_{\text{nuc}} | \phi_q \rangle$ are one-body integrals, $(pq||rs) = \iint \phi_p^*(\mathbf{r}_1) \phi_q(\mathbf{r}_1) \frac{1}{r_{12}} \phi_r^*(\mathbf{r}_2) \phi_s(\mathbf{r}_2) d\mathbf{r}_1 d\mathbf{r}_2$ are two-electron repulsion integrals in chemist's notation (reduced by 8-fold symmetry), $\hat{a}_p^\dagger/\hat{a}_p$ satisfy $\{\hat{a}_p, \hat{a}_q^\dagger\} = \delta_{pq}$ (enforcing Pauli exclusion), and $E_{\text{core}} = E_{\text{nuc}} + E_{\text{fc}}$.

Understanding example: Consider a hypothetical 1-orbital system ($n = 1$, 1 spatial orbital $\rightarrow$ 2 spin-orbitals). Here h_{pq} is a 1×1 scalar h_{00} , $(pq||rs)$ has one value $(00|00)$. The Hamiltonian is $\hat{H} = h_{00}(\hat{a}_0^\dagger \hat{a}_0 + \hat{a}_1^\dagger \hat{a}_1) + \frac{1}{2}(00|00)\hat{a}_0^\dagger \hat{a}_1^\dagger \hat{a}_1 \hat{a}_0 + E_{\text{core}}$. This shows the one-body and two-body structure without multi-orbital hopping.

2. From Fermions to Qubits: Jordan-Wigner and Parity Mapping.

Three mappings are mutually exclusive alternatives. *Jordan-Wigner (JW):*

$$\hat{a}_j^\dagger = \left(\bigotimes_{k=0}^{j-1} \sigma_Z^{(k)} \right) \otimes \sigma_-^{(j)}, \quad \sigma_- = \frac{\sigma_X + i\sigma_Y}{2}. \quad (2)$$

Z-string encodes parity of preceding electrons, producing $(-1)^{n_{\text{before}}}$ sign. *Pauli weight* (non- I count) determines CNOT cost: weight- $w \rightarrow 2(w-1)$ CNOTs.

Understanding example (number operator $\hat{n}_0$): Consider $\hat{n}_0 = \hat{a}_0^\dagger \hat{a}_0 = \frac{1}{4}(X_0 + iY_0)(X_0 - iY_0)$. There's a trap — Pauli multiplication has $XY \neq 0$. Actually $XY = iZ$, $YX = -iZ$, so cross terms give $2Z$. The correct result is $\hat{n}_0 = \frac{I-Z_0}{2}$. This identity is crucial for two-body JW transformations. For off-diagonal terms (like $h_{01}\hat{a}_0^\dagger \hat{a}_1$), one must handle Z-strings and Pauli cross terms — the method is similar but with more steps, left as an exercise.

Parity mapping: Encodes particle number directly as parity — qubit j stores the parity of the first j orbitals' occupation rather than orbital j 's occupation. Key advantage: N_α, N_β conservation manifests as *single-qubit* $Z \rightarrow$ built-in 2-qubit reduction, no extra tapering needed (JW needs additional tapering).

Bravyi-Kitaev (BK) transformation: Distributes parity information across multiple qubits (rather than JW's linear Z-string), reducing Z-string length to $O(\log n)$. Specifically, $\hat{a}_j^\dagger$'s Z-string involves only $O(\log j)$ qubits (selected by binary decomposition), so the maximum Pauli weight is $O(\log n)$.

SQD workflow.

1. **Quantum computer:** Prepare an approximate ground state $|\Psi\rangle$ using a quantum circuit (e.g., HF or LUCJ state).
2. **Quantum computer:** Measure $|\Psi\rangle$ in the computational basis to obtain S bitstrings, each corresponding to an electronic configuration.
3. **Classical computer:** Configuration recovery [Robledo, *et al.*, 2024] — use average occupancy $\bar{n}_i$ to fix particle-number-violating bitstrings. Deduplicate to obtain M unique configurations $\{D_k\}$.
4. **Classical computer:** Subspace diagonalization [Robledo, *et al.*, 2024] — build the CI matrix $H_{kl} = \langle D_k | \hat{H} | D_l \rangle$ in $\mathcal{S} = \text{span}\{D_k\}$ (using Slater-Condon rules), exactly diagonalize to obtain E_{SQD} .
5. **Classical computer:** Update $\bar{n}_i$ with eigenstate amplitudes, repeat steps 2–4 until energy converges.

Figure 1: SQD workflow.

For N_2 ($n=10$ spatial orbitals, 20 qubits), JW’s max weight is 10 (Z-string spans 10 qubits), while BK’s is only $\lceil \log_2 20 \rceil = 5$. BK’s weight advantage is more pronounced for larger molecules, but the mapping rules are more complex than JW.

$\mathbb{Z}_2$ symmetry reduction: Closed-shell molecules have N_α, N_β particle-number conservation, each a $\mathbb{Z}_2$ symmetry (a Pauli operator commuting with $\hat{H}$), eliminating 1 qubit each, 2 total.

Note: SQD solver *forces JW* (CI matrix in Slater determinant basis requires direct bitstring-configuration correspondence). VQE can use any mapping.

3. Hartree-Fock State, Electronic Configurations, and CI Matrix.

HF state: electrons independently occupy lowest-energy orbitals:

$$|\text{HF}\rangle = \prod_{i \in \text{occ}} \hat{a}_i^\dagger |0\rangle. \quad (3)$$

After JW, corresponds to a specific computational basis state $|b_{n-1} \cdots b_0\rangle$. FCI dimension = $\binom{2n}{N_e}$.

Understanding example (2 spin-orbitals, 1 electron): Consider 1 electron in 2 spin-orbitals (0,1). FCI dimension = $\binom{2}{1}=2$, configurations $|01\rangle$ and $|10\rangle$. CI matrix is 2×2 ; off-diagonal $\langle 01 | \hat{H} | 10 \rangle$ is the electron hopping coupling — the corresponding Fock matrix element. *Slater-Condon rules* provide a systematic way to compute these: diagonal = one-body + two-body integral combination; off-diagonal depends on how many orbitals differ (0, 1, or 2).

For HF + singles subspace, the key question is: is the coupling between HF and single excitations zero? This depends on HF self-consistency (Brillouin condition), but the specific derivation is left to the reader.

4. SQD Algorithm and Configuration Recovery.

SQD [Robledo, *et al.*, 2024] samples configurations and diagonalizes in subspace. Variational property: $E_{\text{SQD}} = \min_{|\psi\rangle \in \mathcal{S}} \langle \psi | \hat{H} | \psi \rangle \geq E_0$. Full subspace $\rightarrow E_{\text{SQD}} = E_{\text{FCI}}$.

The SQD workflow is shown as follows.

Configuration recovery concept: Average occupancy $\bar{n}_i = \frac{1}{S} \sum_{s=1}^S b_i^{(s)}$ reflects the “prior probability” of orbital i being occupied. When noise causes bitstrings to violate particle-number con-

straints, $\bar{n}_i$ provides a basis for judging which bit is most likely erroneously flipped.

Understanding example: 4-qubit system, correct state has 2 ones. Sampled 3 bitstrings: $|1010\rangle$ (correct), $|1110\rangle$ (wrong, 3 ones), $|0010\rangle$ (wrong, 1 one). Average $\bar{n} = (1/3, 2/3, 1/3, 0)$. For $|1110\rangle$ need to flip one $1 \rightarrow 0$: bit 0 has lowest $\bar{n}_0 = 1/3 \rightarrow$ flip bit 0 $\rightarrow |0110\rangle$. Why is this strategy optimal? Provable via maximum likelihood — left to the reader.

5. LUCJ Ansatz and Quantum Circuit Decomposition.

LUCJ [Sung, et al., 2023] builds correlated state from CCSD amplitudes:

$$|\Psi_{\text{LUCJ}}\rangle = \prod_{\mu=1}^{n_{\text{reps}}} \hat{U}_{\mu} e^{i\hat{J}_{\mu}} \hat{U}_{\mu}^{\dagger} |\text{HF}\rangle, \quad (4)$$

where $\hat{U} = e^{\hat{\kappa}}$ (orbital rotation from CCSD t_1 , $\hat{\kappa}$ anti-Hermitian $\rightarrow \hat{U}$ unitary) decomposes into Givens rotations (each = 2 CNOT + single-qubit gates), and $e^{i\hat{J}}$ ($\hat{J} = \sum_{ij} J_{ij} \hat{n}_i \hat{n}_j$, from ERIs + CCSD t_2) maps to R_{ZZ} gates (each = 2 CNOT + R_Z). “Local” retains only adjacent-qubit R_{ZZ} . `ccsd_scale` λ : $\lambda=0 \rightarrow \text{HF}$; $\lambda=1 \rightarrow$ approximates CCSD.

Understanding example (single-qubit term $e^{iJ_i \hat{n}_i}$): Using the identity $\hat{n}_i = \frac{I-Z_i}{2}$ from above: $e^{iJ_i \hat{n}_i} = e^{iJ_i/2} \cdot e^{-iJ_i Z_i/2}$. First term is global phase, second is $R_Z(J_i)$ — no CNOT needed. For the two-qubit term $e^{iJ_{ij} \hat{n}_i \hat{n}_j}$, one needs to expand $\hat{n}_i \hat{n}_j = \frac{(I-Z_i)(I-Z_j)}{4}$, then use commuting operators’ exponential separation. The key insight is that the expansion contains a $Z_i Z_j$ term $\rightarrow R_{ZZ}$. Specific derivation left to the reader.

Givens rotation creates superposition: $\hat{U}_{\text{Givens}} |01\rangle = \cos \theta |01\rangle + \sin \theta |10\rangle$ — product state becomes superposition, measurement randomness covers more configurations.

Edge coloring: 1D chain’s adjacent edges can be 2-colored, same-color edges don’t share qubits $\rightarrow$ parallelizable. Full UCJ: $\binom{n}{2}$ pairs, depth $O(n^2)$; LUCJ: $n-1$ adjacent pairs, depth $O(1)$.

6. SQD vs VQE: Two Quantum-Classical Paradigms.

VQE [Peruzzo, et al., 2014] prepares parameterized state $|\Psi(\theta)\rangle$, measures $\langle P_k \rangle$ per Pauli term, optimizes θ . SQD samples bitstrings, diagonalizes classically. VQE quantum cost $O(N^4/\epsilon^2)$; SQD $O(S)$, independent of N . VQE: shallow circuit, flexible ansatz, any mapping. SQD: deep LUCJ circuit, forced JW, LUCJ-only ansatz.

Variational properties compared: VQE energy is the Rayleigh quotient $\langle \Psi | \hat{H} | \Psi \rangle$, variational over the full Hilbert space. SQD energy is the subspace minimum $\min_{|\phi\rangle \in \mathcal{S}} \langle \phi | \hat{H} | \phi \rangle$. Both satisfy $\geq E_0$. The energy accuracy relationship — is SQD necessarily no worse? The key observation is that $|\Psi\rangle$ itself lies in $\mathcal{S}$ (defined by $|\Psi\rangle$ ’s support), from which their relationship can be derived. Specific proof left to the reader.

VQE parameter-shift: $\frac{\partial E}{\partial \theta_k} = \frac{E(\theta_k + \pi/2) - E(\theta_k - \pi/2)}{2}$, needs $2p$ evaluations/step. *Circuit fidelity:* $F \approx (1 - p_{2q})^{N_{2q}}$ — deeper circuits, lower fidelity. This is SQD’s main risk on NISQ hardware.

7. EWF Embedding: Schmidt Decomposition and Bath Construction.

DMET [Knizia & Chan, 2012] uses Schmidt decomposition:

$$|\Psi\rangle = \sum_{k=1}^r \lambda_k |u_k\rangle \otimes |v_k\rangle, \quad \lambda_k \geq 0, \quad \sum_k \lambda_k^2 = 1. \quad (5)$$

λ_k (descending) measure impurity-environment entanglement. Retain n_{bath} largest- λ_k environment states as bath. Schmidt rank $r \leq \min(\dim \mathcal{H}_{\text{imp}}, \dim \mathcal{H}_{\text{env}}) \rightarrow \text{bath upper bound} = n_{\text{imp}}$.

Projector matrices and Schmidt rank: HF density matrix γ^{HF} is a *projector* — satisfying $\gamma^2 = \gamma$ (idempotent), with N_e eigenvalues of 1 and the rest 0. An important property of projectors is that their Schmidt rank doesn’t exceed either subsystem’s dimension. From this, one can derive that DMET minimal bath is exact at HF level — but the specific proof is left to the reader. Correlated $\gamma^{\text{corr}} \neq (\gamma^{\text{corr}})^2$ (non-projector), Schmidt rank may exceed $n_{\text{imp}} \rightarrow$ minimal bath insufficient.

EWF [Nusspickel & Booth, 2022] adds MP2 bath natural orbitals (BNO): compute MP2 density correction $\rightarrow$ diagonalize for natural orbitals $\rightarrow$ truncate at threshold η . DMET bath captures *static correlation* (orbital near-degeneracy, bond breaking); MP2 bath captures *dynamic correlation* (instantaneous electron fluctuations).

Large molecule fragmentation data: C_2H_4 (28 qubits $\rightarrow$ max fragment 26); C_6H_6 benzene (72 $\rightarrow$ 30); CH_3COOH acetic acid (88 $\rightarrow$ 50); C_2H_4 cc-pVDZ (96 $\rightarrow$ 44) — all feasible on NISQ after EWF.

Physical picture at strong correlation: At equilibrium $\lambda_1 \approx 1$ (dominant). At bond breaking forms Bell pair: $\lambda_1 \approx \lambda_2 \approx 1/\sqrt{2}$ — spectrum flattens, effective Schmidt rank increases $\rightarrow$ DMET minimal bath truncation error surges.

8. Energy Reconstruction: Eliminating Double-Counting.

Standard reconstruction:

$$E_{\text{total}} = E_{\text{MF}} + \sum_f (E_f - E_{\text{MF},f}). \quad (6)$$

Why it eliminates double-counting: The key is which terms cancel in the difference $(E_f - E_{\text{MF},f})$. If one decomposes the fragment energy into “an environment-independent core part” and “a solver-dependent electronic part”, the core part is the same in E_f and $E_{\text{MF},f}$ — it naturally cancels. What remains is only “the solver’s improvement over HF”, i.e., $\Delta E_{\text{corr},f}$. The specific algebraic derivation is left to the reader. Correlation energy is local, fragments don’t overlap $\rightarrow$ no double-counting.

Numerical example: H_2O O fragment $E_{\text{O}} = -75.010$, $E_{\text{MF},\text{O}} = -74.980 \rightarrow$ correction = -0.030 Ha. H_2 fragment correction = -0.022 Ha. Total = $-74.963 + (-0.030) + (-0.022) = -75.015$ Ha $\approx$ FCI. All fragments with HF $\rightarrow$ correction = 0 $\rightarrow E_{\text{total}} = E_{\text{MF}}$.

Error balancing: Total error includes fragment truncation ϵ_{frag} (decreases with larger fragments) and sampling error ϵ_{SQD} (decreases with more samples), competing. Under a fixed budget, how to allocate to minimize total error? This is a constrained optimization problem — the specific derivation is left to the reader. Intuitively, total accuracy is determined by the weakest link (“barrel effect”).

9. EWF Software Toolchain and Module Description.

The practical implementation of EWF involves the collaboration of multiple software packages. The following subsections detail each package’s role, the orchestration pattern, data interface, and end-to-end workflow.

(a) *Software packages.*

Package	Role	Key modules
PySCF	Electronic structure	RHF/ROHF, MO generation, h_{pq} / $(pq rs)$ extraction, FCI/CCSD/MP2 solvers
Vayesta	Quantum embedding	<code>vayesta.ewf.EWF</code> (DMET+MP2 bath, IAO fragmentation), <code>vayesta.dmet.DMET</code> (Schmidt minimal bath)
TensorCircuit	Quantum circuit	LUCJ state preparation, bitstring sampling, noise simulation

Bath type is controlled by the `bath_type` parameter ("`dmet`" or "`mp2`"); BNO truncation threshold by `threshold`. The EWF workflow starts from PySCF's Mean-Field calculation; the output density matrix and orbital coefficients are passed to the Vayesta embedding layer, which produces per-fragment active-space integrals.

(b) *Orchestration pattern.* The above packages are assembled into a complete pipeline via the Registry Pattern:

- *Embedder registry:* manages embedding methods (EWF / DMET / lightweight PySCF), creates embedder instances by method name and parameters
- *Solver registry:* manages solvers (FCI / CCSD / MP2 / SQD / VQE), creates solver instances by type name and parameters
- *Pipeline orchestrator:* chains "embed $\rightarrow$ solve $\rightarrow$ reconstruct" steps

Pseudocode.

1. Create embedder and solver via registry:

```
embedder = create_embedder("ewf", bath_type="mp2", threshold=1e-6)
solver = create_solver("sqd", n_samples=20000)
```
2. Assemble pipeline and run:

```
pipeline = Pipeline(embedder=embedder, solver=solver)
result = pipeline.run(molecule="h2o")
# -> result.total_energy, result.fragments
```

(c) *FragmentData interface.* The standard data container between embedding and solver layers:

Field	Shape	Description
<code>heff</code>	$(n_{\text{orb}}, n_{\text{orb}})$	1e Hamiltonian with embedding potential
<code>eris</code>	(n_{orb}^4)	2e integrals, chemist notation $(pq rs)$
<code>norb</code>	scalar	Active space orbital count
<code>nocc</code>	scalar	Occupied orbital count
<code>ecore</code>	scalar	Core energy offset

Derived properties: $n_{\text{qubits}} = 2 n_{\text{orb}}$, n_{elec}^{α} , n_{elec}^{β} . Solvers solve independently through this interface without accessing original molecule data.

(d) *End-to-end workflow.* The complete EWF-SQD pipeline:

- Step 1. RHF:** PySCF performs RHF on the full molecule $\rightarrow$ MO coefficients, density matrix.
- Step 2. Fragmentation:** Vayesta performs IAO fragmentation + Schmidt/MP2 bath construction $\rightarrow$ set of `FragmentData` (embedding potential folded into `heff`).
- Step 3. Quantum sampling:** For each fragment, TensorCircuit/Qiskit builds the LUCJ circuit from CCSD amplitudes, samples S bitstrings.
- Step 4. SQD diagonalization:** Configuration recovery $\rightarrow$ CI matrix in sampled subspace $\rightarrow$ exact diagonalization $\rightarrow$ per-fragment energy E_f .
- Step 5. Energy reconstruction:** $E_{\text{total}} = E_{\text{MF}} + \sum_f (E_f - E_{\text{MF},f})$, eliminating double-counting.

The key advantage: each fragment's qubit count $2n_{\text{frag}} \ll 2N$ (full molecule), making previously infeasible molecules tractable on NISQ hardware.

(e) *Minimal example: PySCF + TensorCircuit + SQD.* The following pseudocode illustrates the full quantum-classical loop on H_2 (4 qubits). TC prepares a correlated quantum state and samples bitstrings; the classical side performs configuration recovery, builds the CI matrix via Slater-Condon rules, and diagonalizes to obtain the SQD energy. For H_2 , the full FCI subspace has only 4 determinants, so SQD recovers FCI exactly.

Pseudocode.

```

1. PySCF:  $\text{H}_2$  + RHF  $\rightarrow$  MO-basis integrals:
mol = pyscf.gto.M(atom="H 0 0 0; H 0 0 0.74", basis="sto-3g")
mf = pyscf.scf.RHF(mol).run()
h1e = mf.mo_coeff.T @ mf.get_hcore() @ mf.mo_coeff
eri = np.einsum('pqrs,pi,qj,rk,sl->ijkl',
    mol.intor('int2e_sph'), *[mf.mo_coeff]*4)

2. TC: prepare HF state + entangling gates, sample:
c = tc.Circuit(nq); c.x(0); c.x(norb)
c.cnot(0,1); c.cnot(norb, norb+1)
c.ry(1, theta=0.3); c.ry(norb+1, theta=0.3)
bitstrings = c.sample(batch=1000, format="count_dict_int")

3. Configuration recovery: filter bitstrings by Hamming weight =  $n_e$ , augment with
HF + singles + doubles.
basis = recover_configurations(bitstrings, nelec, nq)

4. SQD: CI matrix via Slater-Condon + diagonalize:
H = build_ci_matrix(basis, h1e, eri, ecore)
E_sqd = np.linalg.eigvalsh(H)[0]

5. Compare:
fci = pyscf.fci.FCI(mf).run()
print(f"E(HF)={mf.e_tot:.8f} E(FCI)={fci.e_tot:.8f}")
print(f"E(SQD)={E_sqd:.8f}")
Output: E(HF)=-1.11675931, E(FCI)=-1.13728383, E(SQD)=-1.13728383

```

Here `recover_configurations` filters bitstrings by Hamming weight = n_e and augments with symmetry-allowed excitations; `build_ci_matrix` applies Slater-Condon rules to compute $\langle D_i | \hat{H} | D_j \rangle$

from h_{pq} and $(pq|rs)$. For larger molecules, insert Vayesta EWF fragmentation between steps 1 and 2; the rest applies per-fragment.

Problem 3: Quantum Approximate Optimization Algorithm for MaxCut Problem

The Quantum Approximate Optimization Algorithm (QAOA) is implementable on near-term quantum computers and is widely regarded as a promising candidate for demonstrating quantum advantage. **In this problem, you will use QAOA to solve Max-Cut, an NP-hard combinatorial optimization problem.**

MaxCut problem. The MaxCut problem is a classical combinatorial optimization problem defined on an undirected graph. Given a graph $G = (V, E)$, the goal is to divide the vertex set V into two disjoint subsets such that the number of edges crossing between the two subsets is maximized.

QAOA workflow. Before specializing to MaxCut, QAOA can be formulated for a generic Boolean maximization problem. Let $z = z_1 z_2 \dots z_n \in \{0, 1\}^n$ and define an objective function

$$C(z) = \sum_{k=1}^m C_k(z), \quad C_k(z) \in \{0, 1\}. \quad (7)$$

Each clause C_k checks whether the bitstring, or a substring of it, satisfies a certain property. The goal is to find

$$z^* = \arg \max_{z \in \{0,1\}^n} C(z). \quad (8)$$

QAOA promotes C to a diagonal quantum operator by defining

$$C |z\rangle = C(z) |z\rangle. \quad (9)$$

Note that C has eigenvectors $|z\rangle$ and eigenvalues $C(z)$.

We now describe the QAOA workflow. Define the problem operator

$$U(C, \gamma) = e^{-i\gamma C}. \quad (10)$$

When the clause operators commute, this can be decomposed as

$$U(C, \gamma) = \prod_{k=1}^m e^{-i\gamma C_k}. \quad (11)$$

Also define the mixing operator

$$U(B, \beta) = e^{-i\beta B} = \prod_{j=0}^{n-1} e^{-i\beta X_j}, \quad (12)$$

where $B = \sum_{j=0}^{n-1} X_j$. Starting from the state

$$|s\rangle = |+\rangle^{\otimes n}, \quad (13)$$

a depth- p QAOA state is

$$|\gamma, \beta\rangle = U(B, \beta_p) U(C, \gamma_p) \dots U(B, \beta_1) U(C, \gamma_1) |s\rangle, \quad (14)$$

where $\gamma = (\gamma_1, \dots, \gamma_p)$ and $\beta = (\beta_1, \dots, \beta_p)$. The associated objective is

$$F_p(\gamma, \beta) = \langle \gamma, \beta | C | \gamma, \beta \rangle. \quad (15)$$

The QAOA workflow is summarized below (Figure 2). See details in [Blekos, *et al.*, 2023].

We consider three graphs (Figure 3).

QAOA workflow.

1. **Quantum computer:** Prepare an initial state $|\psi\rangle = |s\rangle$.
2. **Classical computer:** Initialize the $2p$ parameters γ and β . If no better choice is available, they may be initialized to zero.
3. **Quantum computer:** Construct $|\psi_p\rangle = |\gamma, \beta\rangle$ using Eq. (14).
4. **Quantum computer:** Measure $|\psi_p\rangle = |\gamma, \beta\rangle$ in the computational basis and obtain a bitstring z .
5. **Classical computer:** Evaluate $C(z) = \sum_{k=1}^m C_k(z)$.
6. Repeat steps 1–4 N_{shot} times. The resulting samples estimate the distribution over computational-basis states $|z\rangle$. Each sampled bitstring has an associated objective value $C(z)$; from these samples, record the best observed value $C(z'')$ and its corresponding bitstring z'' , and estimate the expectation value $F_p(\gamma, \beta) = \langle \psi_p | C | \psi_p \rangle$.
7. Update the $2p$ parameters γ and β , and repeat steps 3–6. Across optimization iterations, keep track of the best observed value $C(z'')$ and output the corresponding bitstring. The stopping criterion depends on the chosen classical optimizer. Thus, QAOA can be viewed as a $2p$ -parameter optimization problem in which each function evaluation requires quantum-state preparation and measurement.

Figure 2: QAOA workflow.

- (a) C_4 : 4 vertices and edges $[(0, 1), (1, 2), (2, 3), (0, 3)]$;
- (b) G_6 : 6 nodes and edges $[(0, 1), (3, 4), (2, 5), (0, 3), (4, 5), (1, 2), (1, 4)]$;
- (c) G_9 : 9 nodes and edges $[(0, 1), (3, 4), (7, 8), (2, 5), (0, 3), (4, 5), (6, 7), (1, 2), (4, 7), (5, 8), (3, 6), (1, 4)]$.

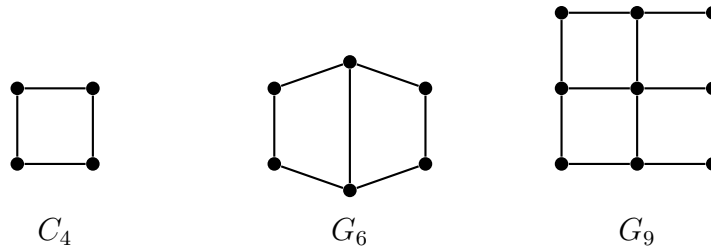


Figure 3: Three graph instances

Demonstrate numerically that QAOA can be used to solve MaxCut on graphs C_4 , G_6 and G_9 . Please do the following using TensorCircuit:

1. Define the MaxCut Objective

For any unweighted graph $G = (V, E)$ with n vertices, formulate the MaxCut problem as the objective function $C(z)$ for QAOA, and write the corresponding Hamiltonian C . Prove that for every computational basis state $|z\rangle$,

$$C |z\rangle = C(z) |z\rangle. \quad (16)$$

2. QAOA-MaxCut Warm-up

Let $p = 1$ and $N_{shot} = 100$ and choose an optimizer you're familiar with as the classical optimizer for QAOA. Following the QAOA workflow shown in Figure 2, use QAOA to solve MaxCut on graphs C_4 , G_6 , and G_9 . More specifically, you should

- (a) Prepare the initial state $|s\rangle$;
- (b) Construct the parameterized quantum circuit $U(B, \beta_1)U(C, \gamma_1)$ for QAOA and prepare the quantum state $|\psi_1\rangle = U(B, \beta_1)U(C, \gamma_1)|s\rangle$.
- (c) Prepare N_{shot} copies of $|\psi_1\rangle$ and measure them to obtain N_{shot} bitstrings z . From the measured bitstrings $z^{(1)}, \dots, z^{(N_{shot})}$, find $z'' = \arg \max C(z)$ and compute $C(z'')$. Use these N_{shot} bitstrings to estimate the average value of C , namely $F_1 = \langle \psi_1 | C | \psi_1 \rangle$.
- (d) Update the two parameters γ_1 and β_1 using the classical optimizer and the estimated average value of C .

3. Increasing the QAOA Depth p

Run your QAOA-MaxCut in Problem 3.2 **QAOA-MaxCut warm-up** for different QAOA depths p . Investigate how the QAOA depth p affects algorithm performance — including, but not limited to, runtime, iteration count, and the MaxCut value obtained.

4. Initialization Strategy

In the QAOA workflow of Figure 2, all circuit parameters are initialized to zero. Many initialization strategies exist: for example, zero initialization, uniform random initialization, and linear-ramp / annealing-inspired initialization. Please survey these strategies and run QAOA-MaxCut on Problem 3.2 **QAOA-MaxCut warm-up** with different initialization schemes.

5. Optimizer Choice

In QAOA-MaxCut, the choice of classical optimizer strongly affects performance and depends on factors such as the graph type and the number of parameters. Run QAOA-MaxCut in Problem 3.2 **QAOA-MaxCut warm-up** using different classical optimizers — for example, grid-based methods, gradient-based methods (Gradient Descent, Adam, BFGS, L-BFGS-B), and gradient-free methods (COBYLA). You are not limited to the optimizers listed above — feel free to survey additional ones if interested. When running QAOA-MaxCut, choose an optimizer suited to the input graph and the number of parameters.

6. Different Shot Numbers

Run QAOA-MaxCut in Problem 3.2 **QAOA-MaxCut warm-up** with different shot numbers N_{shot} .

7. Revisited QAOA-MaxCut

In Problems 3.2–3.6, we explored improvements to QAOA-MaxCut along different dimensions. Now, combine these strategies so that QAOA-MaxCut on graphs C_4 , G_6 , and G_9 achieves the best overall performance, including, but not limited to, low runtime, few iterations, and a high MaxCut value.

8. Experimental

Running QAOA-MaxCut on current quantum computers is challenging due to their limited qubit coherence times and high error rates. Despite these challenges, we will execute QAOA-MaxCut on graph C_4 on superconducting quantum hardware.

Hints:

- (a) The fidelity of single-qubit and two-qubit gates generally depends on which qubits they act on. Higher fidelity means the gate more closely matches the ideal unitary.
- (b) The larger the depth and size of a quantum circuit, the greater the accumulated error.
- (c) Qubits are subject to connectivity constraints, so two-qubit gates cannot be applied directly to arbitrary qubit pairs. Choose nearby qubits to minimize additional gate overhead.

9. Exploratory Question (Open)

In Problems 3.2–3.8, we have implemented the complete QAOA-MaxCut workflow. However, these problems focused only on standard QAOA applied to a few small, highly structured graphs. In this part, you are encouraged to go beyond this setting: implement QAOA and its variants on different classes of input graphs, and investigate theoretical aspects of their trainability (See details in [Blekos, *et al.*, 2023]). Specifically,

- (a) (Graph families) The performance of QAOA-MaxCut can depend strongly on the structure of the input graph, such as its degree distribution, density, symmetry, bipartiteness, and the presence of frustrated cycles. Therefore, to understand how robust QAOA is across different MaxCut instances, it is useful to test the algorithm on several representative graph families. You may consider larger graphs with specific structural properties, such as symmetric graphs (paths, cycles, complete graphs, bipartite graphs), d -regular graphs (especially 3-regular graphs), Erdős–Rényi random graphs $G(n, p)$, hardware-connectivity graphs, and weighted graphs.
- (b) (QAOA ansatz and its variants) The performance of QAOA-MaxCut can also depend strongly on the choice of ansatz, since different ansatz structures determine the expressibility of the variational circuit, the number of trainable parameters, the circuit depth, and the difficulty of classical optimization. Therefore, to understand the role of ansatz design in QAOA, it is useful to compare the standard QAOA ansatz with several representative variants. You may try ansätze with different parameter structures or mixer designs, such as the standard QAOA with one global γ_l and one global β_l per layer, multi-angle QAOA with edge-dependent $\gamma_{ij,l}$ and vertex-dependent $\beta_{i,l}$, warm-start QAOA with a problem-informed initial state and modified mixer, QAOA with alternative mixers, hardware-efficient QAOA-inspired ansätze, and recursive or adaptive QAOA variants. Compare how these choices affect the optimized MaxCut value, parameter count, circuit depth, and optimization stability.
- (c) (Classical optimization) After the QAOA circuit ansatz is fixed, the next key challenge is to find good variational parameters efficiently. Since the QAOA objective landscape may contain local optima and can be affected by initialization, optimizer choice, shot noise, and stopping criteria, the training procedure plays an important role in the final MaxCut performance. You may try different optimizers, such as grid search, gradient descent, Adam, Adagrad, COBYLA, Powell, and SPSA; different initialization strategies, such as zero initialization, small random initialization, multiple random restarts, layerwise initialization, and linear-ramp initialization; and different training settings, such as exact expectation evaluation versus finite-shot estimation. Compare the optimized MaxCut value, convergence speed, number of function evaluations, sensitivity to random seeds, and stability under sampling noise.
- (d) (Dynamical Lie algebra and trainability) In this part, you will explore how the algebraic structure of the dynamical Lie algebra (DLA) associated with QAOA-MaxCut may

affect the trainability of the QAOA ansatz. Roughly speaking, for the MaxCut problem Hamiltonian C and the mixer Hamiltonian B , the DLA is generated by repeatedly taking commutators such as $[iC, iB]$, $[iC, [iC, iB]]$, and so on. The linear span of all independent operators obtained in this way forms the DLA. In QAOA-MaxCut, the variance of the optimization objective can be fully characterized by the DLA; see [Ragone *et al.*, 2024] for details. Since different input graphs generally give rise to different DLAs, numerically investigate how the dimension and algebraic structure of the DLA vary across input graphs, and further study how these variations relate to the trainability of QAOA-MaxCut. (Please see more details in [Allcock, *et al.*, 2026] for DLA and [Brian C. Hall, 2003] for Lie algebra.)